

미션 데이터 흐름 설계서

Tablet → command_server → mission_control_node → Nav2 → 차량

대상 패키지: `combatrobot_1/robot_server` · `combat_nav2/combat_robot_nav2` | 메시지 패키지: `combat_robot_msgs` | 작성일: 2026-06-01 | ROS 2 Jazzy / Nav2

1. 개요

태블릿 앱이 그린 GPS 경로(웨이포인트 목록)를 차량이 자율주행으로 순회하기까지의 전체 데이터 흐름을 정의한다. 경로 명령은 태블릿에서 TCP 바이너리 프로토콜로 차량 측 `command_server` (combatrobot_1)에 전달되고, ROS 토픽 `/swarm/path_command` 로 변환되어 `mission_control_node` (combat_nav2)로 넘어간다. 이 노드가 위경도를 `/fromLL` 서비스로 map 좌표로 변환하고, Nav2의 `NavigateToPose` 액션으로 웨이포인트를 하나씩 순차 주행시킨다. 최종 속도 명령은 `/cmd_vel` 로 차량 모터(CAN)에 전달된다.

명령 체계는 `LOAD_PATH` `START` `PAUSE` `RESUME` `STOP` 5종이며, 본 문서 기준으로 `PAUSE/RESUME` 까지 `mission_control_node` 에 구현 완료되었다.

2. 시스템 아키텍처

컴포넌트 · 통신 채널 · 메시지 타입 전체도. 번호 ①~⑤는 데이터가 흐르는 순서다.

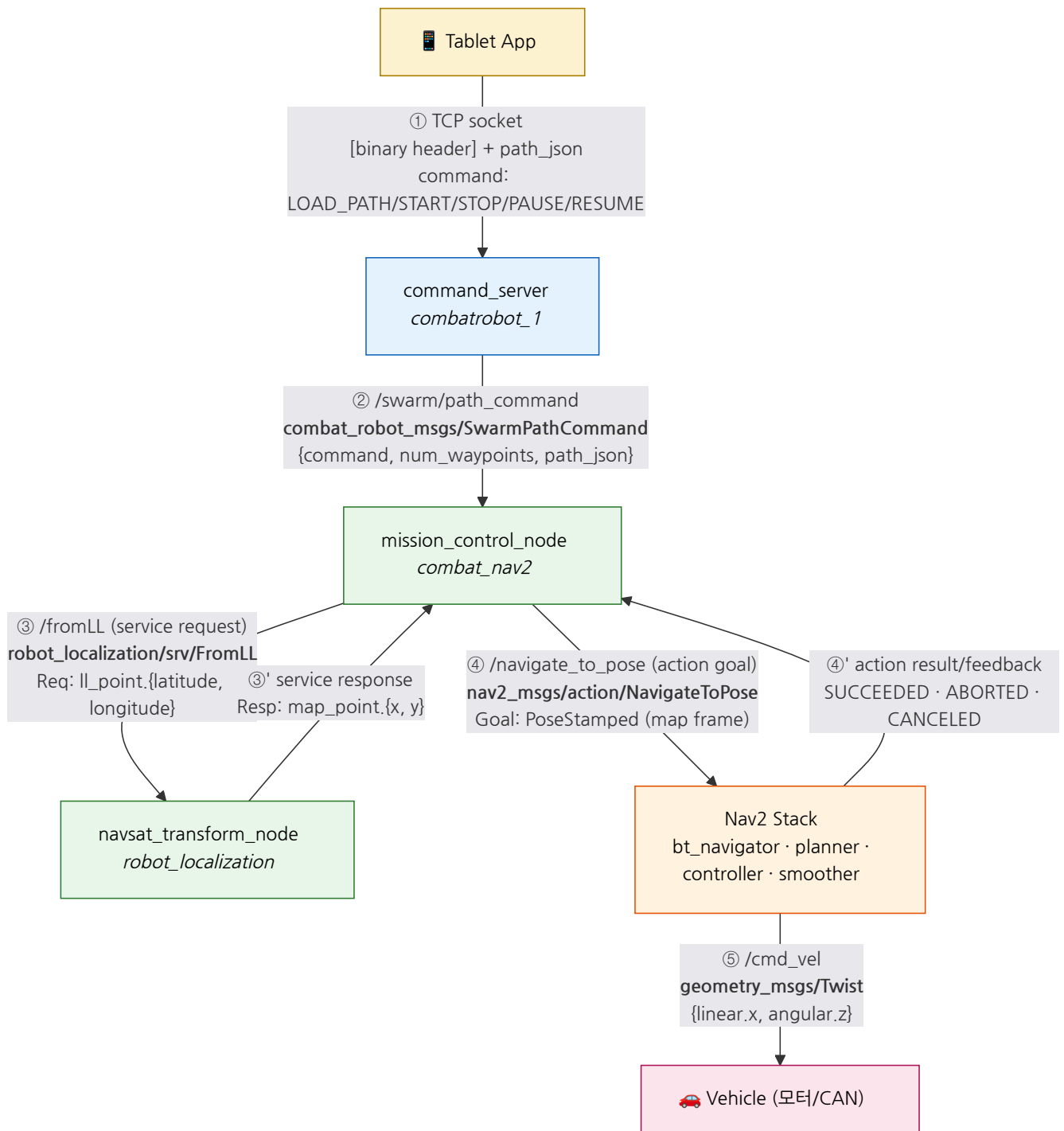


그림 1. 컴포넌트/채널/메시지 타입 전체 흐름

#	구간	채널 / 방식	메시지 타입
①	Tablet → command_server	TCP socket (PATH_PORT 65436)	AssaultCommandHeader(7B) + path_json
②	command_server → mission_control_node	ROS 토픽 /swarm/path_command	combat_robot_msgs/SwarmPathCommand
③	mission_control_node ↔ navsat_transform	ROS 서비스 /fromLL	robot_localization/srv/FromLL
④	mission_control_node → Nav2	ROS 액션 /navigate_to_pose	nav2_msgs/action/NavigateToPose
⑤	Nav2 → 차량	ROS 토픽 /cmd_vel	geometry_msgs/Twist

3. TCP 프로토콜 (Tablet → command_server)

태블릿은 PATH_PORT(65436)로 접속해, 고정 7바이트 헤더 뒤에 (LOAD_PATH 일 때만) JSON 페이로드를 붙여 전송한다. 헤더는 #pragma pack(1) 로 패딩 없이 직렬화된다.

3.1 AssaultCommandHeader (7 bytes, packed)

오프셋	필드	타입	크기	설명
0	command	uint8	1	명령 코드 (아래 표)
1	num_waypoints	uint16	2	웨이포인트 개수 (페이로드와 교차 검증)
3	data_length	uint32	4	뒤따르는 path_json 바이트 수 (없으면 0)

3.2 명령 코드 (AssaultCmdType)

값	명령	페이로드	command_server 처리
1	CMD_START	없음	path_loaded 확인 → 미션 MOVING, 홈 좌표 캡처
2	CMD_STOP	없음	미션 실행 리셋, 상태 NONE
3	CMD_PAUSE	없음	MOVING 일 때만 PAUSED, 속도 0
4	CMD_RESUME	없음	path_loaded 확인 → MOVING 복귀
5	CMD_LOAD_PATH	path_json	웨이포인트 파싱·개수 검증·캐싱 후 발행

검증 게이트: command_server는 num_waypoints (헤더)와 path_json에서 추출한 실제 개수가 다르면 WAYPOINT_COUNT_MISMATCH 로 거절하고 발행하지 않는다. 또한 START/RESUME은 경로 미적재 시 PATH_NOT_LOADED 로 거절한다. 검증을 통과한 명령만 /swarm/path_command 로 재발행된다.

4. ROS 메시지 스키마

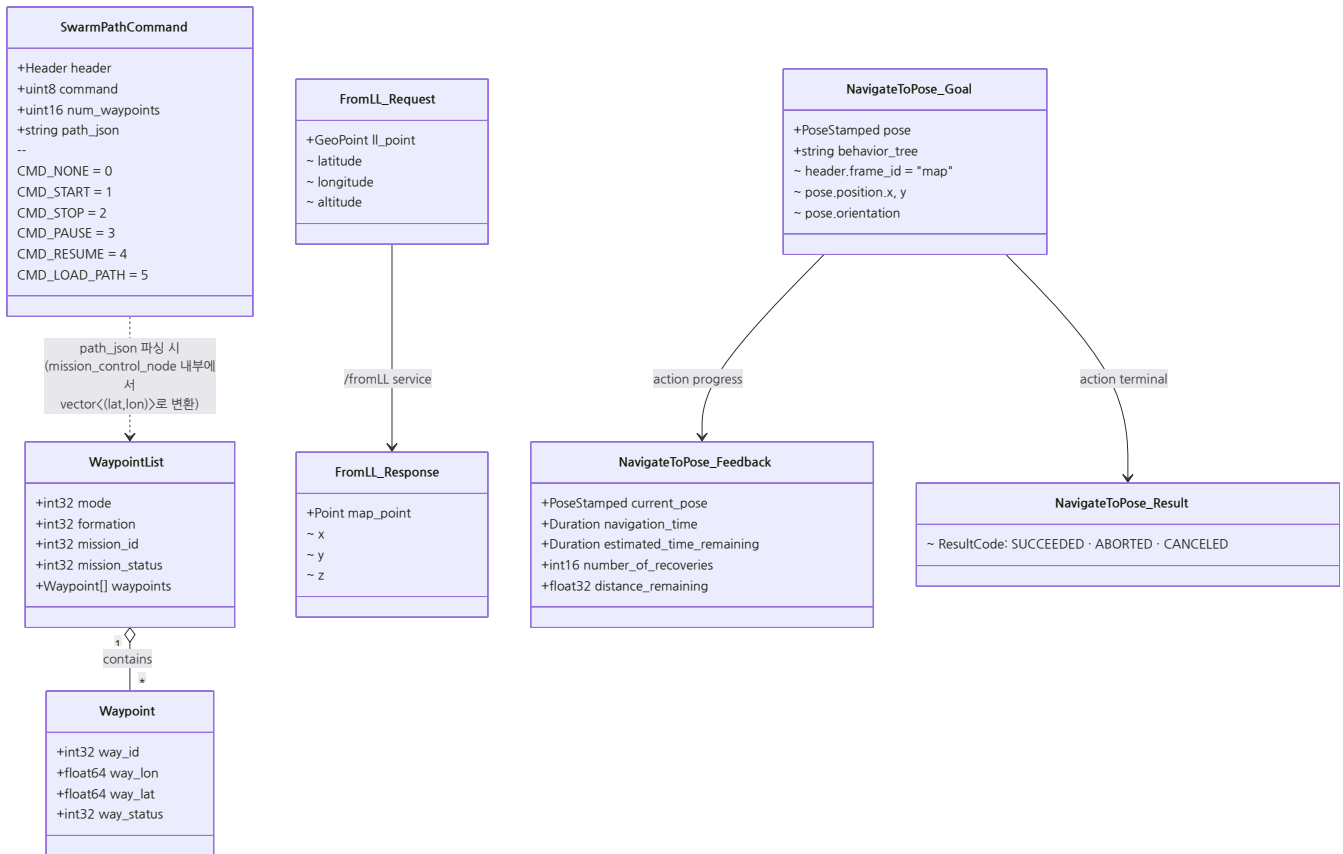


그림 2. 채널별 메시지 스키마

4.1 SwarmPathCommand (combat_robot_msgs)

combatrobot_1 측과 combat_nav2 측의 `.msg` 정의는 완전히 동일(diff 일치)하다.

```

std_msgs/Header header
uint8 CMD_NONE = 0
uint8 CMD_START = 1
uint8 CMD_STOP = 2
uint8 CMD_PAUSE = 3
uint8 CMD_RESUME = 4
uint8 CMD_LOAD_PATH = 5
uint8 command          # 위 상수 중 하나
uint16 num_waypoints    # 웨이포인트 개수
string path_json        # LOAD_PATH 일 때만 채워짐
  
```

4.2 path_json 지원 포맷

`mission_control_node`의 `parsePathJson()`는 의존성 없는 경로 파서로 3가지 포맷을 지원한다.

포맷	예시
객체 배열 (waypoints 키)	<code>{"waypoints":[{"lat":37.4,"lon":126.7}, ...]}</code>
GeoJSON coordinates	<code>{"coordinates":[[126.7,37.4], ...]}</code> (lon,lat 순)
최상위 배열	<code>[{"lat":37.4,"lon":126.7}, ...]</code>

키 별칭 지원: 위도 `lat` / `latitude` , 경도 `lon` / `lng` / `longitude`

5. 정상 / PAUSE·RESUME / STOP 시퀀스

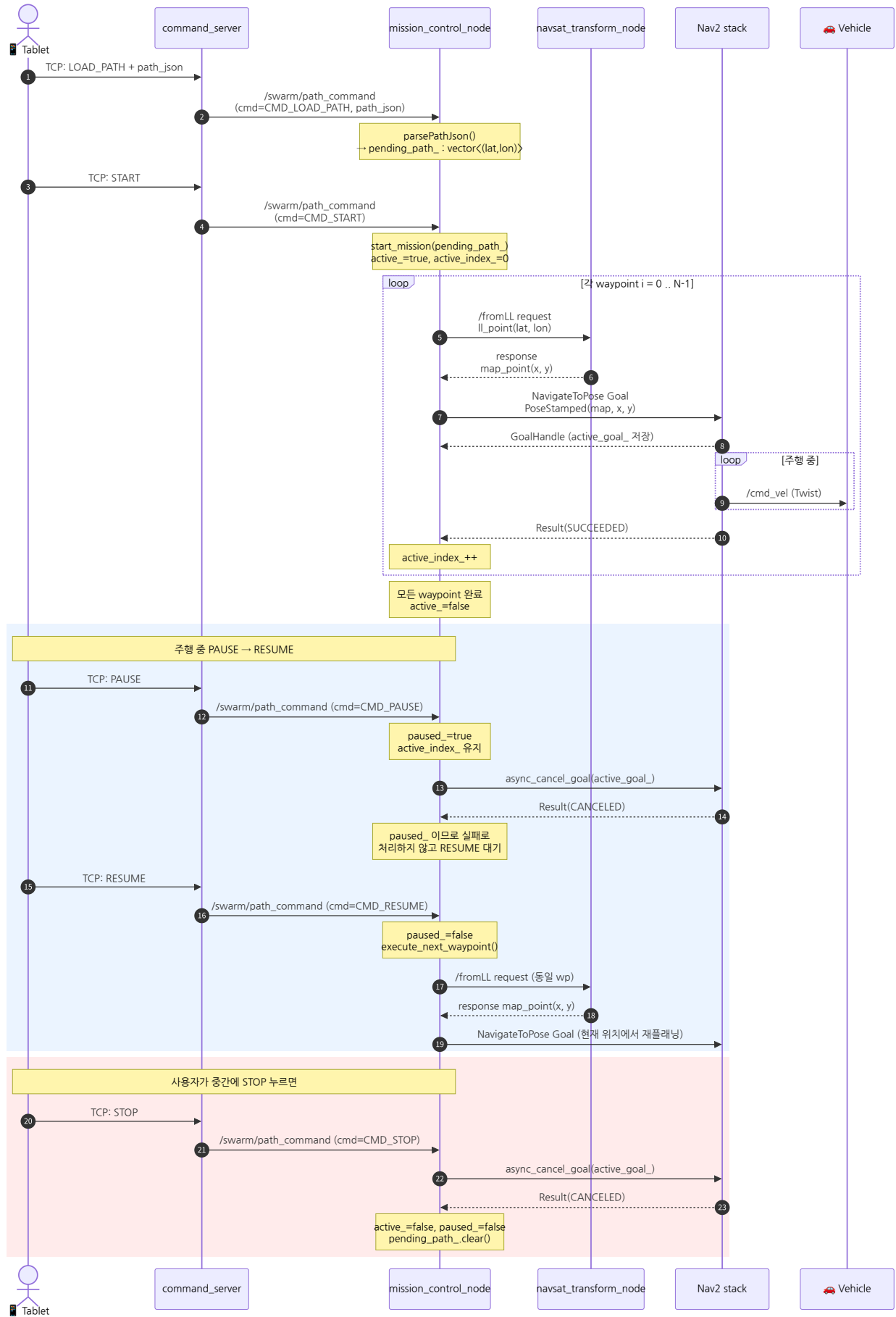


그림 3. LOAD_PATH→START 주행, 중간 PAUSE→RESUME, STOP 시나리오

- 경로 적재: LOAD_PATH → `parsePathJson()` → `pending_path_` 에 캐싱 (주행 시작 안 함)

- 미션 시작: START → `start_mission()` , `active_index_=0` , 웨이포인트 루프 진입
- 웨이포인트 루프: `/fromLL` 로 좌표 변환 → `NavigateToPose` 전송 → SUCCEEDED 시 `active_index_++` → 다음 wp
- 일시정지: PAUSE → 현재 goal cancel(인덱스 유지), RESUME → 현재 wp부터 재플래닝하여 재개
- 중단: STOP → goal cancel, 상태 전체 리셋(`pending_path_` 포함)

6. mission_control_node 상태머신

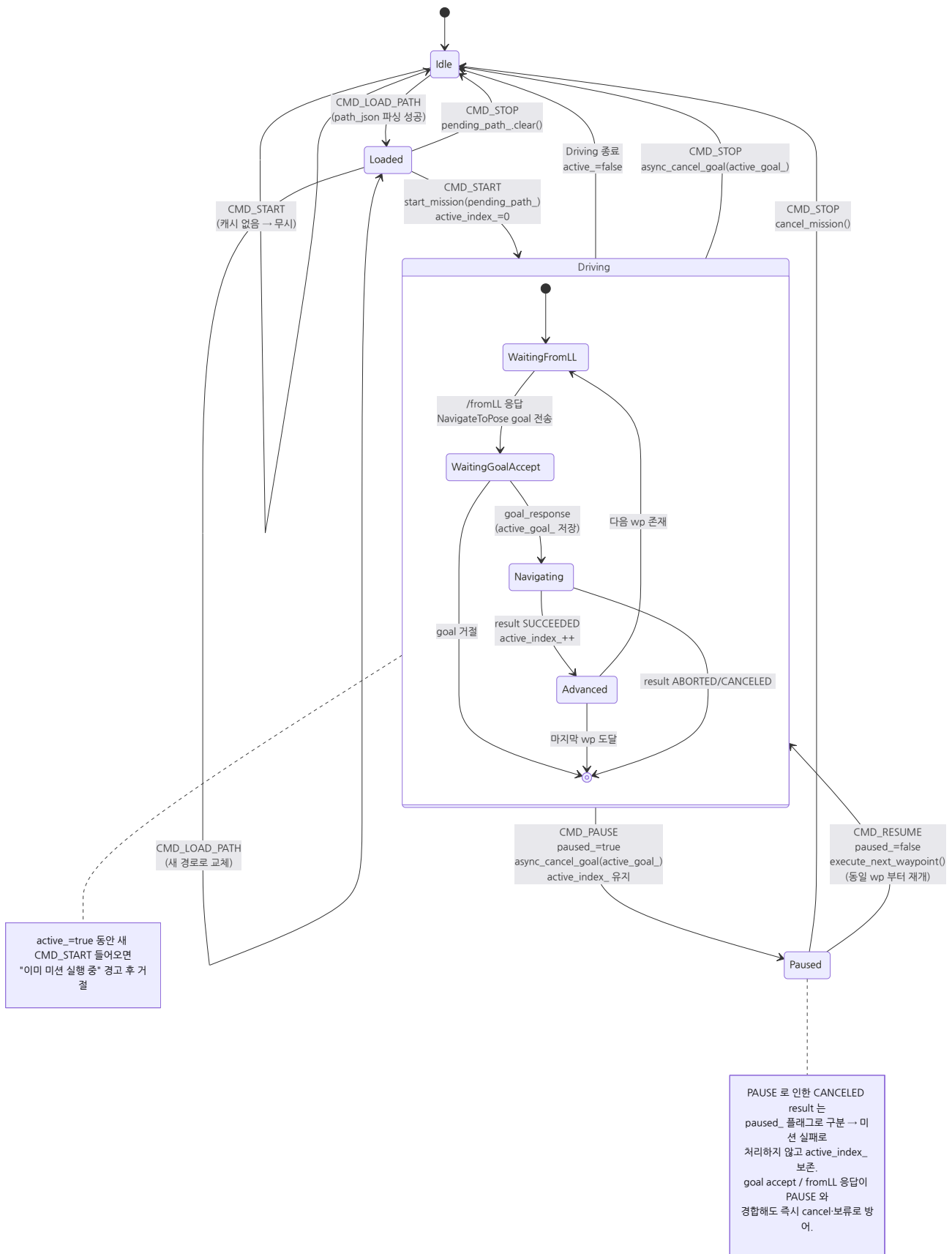


그림 4. 내부 상태머신 (PAUSE/RESUME 반영)

7. PAUSE / RESUME 구현 상세

Nav2 `NavigateToPose` 액션에는 네이티브 "일시정지"가 없다. 따라서 다음 전략을 택했다.

설계 원칙: PAUSE = 현재 goal 을 cancel(웨이포인트 인덱스는 보존), RESUME = 보존된 인덱스의 웨이포인트로 goal 재전송. 단순히 `/cmd_vel` 만 막는 방식과 달리 Nav2가 "주행 중"이라고 착각하지 않고, 재개 시 현재 위치에서 다시 경로를 계획하므로 더 견고하다.

7.1 핵심 상태 변수

기존 `active_` (미션 실행 중)에 더해 `paused_` 플래그를 추가했다. PAUSE 로 발생하는 `CANCELED` 결과를 미션 실패(STOP)와 구분하는 것이 핵심이다.

```
bool active_ = false; // 미션 실행 중 (PAUSE 중에도 true 유지)
bool paused_ = false; // PAUSE 로 현재 goal 을 cancel 한 상태 (active_index_ 유지)
```

7.2 명령 처리

```
case CMD_PAUSE:
    if (!active_ || paused_) return; // 주행 중 아님 / 이미 정지
    paused_ = true;
    if (active_goal_) nav_client->async_cancel_goal(active_goal_);
    break; // active_index_ 는 유지

case CMD_RESUME:
    if (!active_ || !paused_) return; // 미션 없음 / 정지 상태 아님
    paused_ = false;
    execute_next_waypoint(); // 동일 wp 부터 재개
    break;
```

7.3 경합(race) 방어

PAUSE 가 비동기 콜백 경계 어디에서 도착해도 안전하도록 세 콜백에 가드를 추가했다.

PAUSE 도착 시점	처리 (방어 코드)
<code>/fromLL</code> 응답 대기 중	<code>from_ll_response_callback</code> : <code>paused_</code> 면 goal 전송 보류 (RESUME 시 재요청)
goal 전송~accept 사이	<code>goal_response_callback</code> : <code>paused_</code> 면 accept 된 goal 즉시 cancel
주행 중 (goal active)	<code>result_callback</code> : <code>paused_</code> 면 CANCELED 를 실패로 처리하지 않고 RESUME 대기

STOP 과의 차이: STOP은 `cancel_mission()` 에서 `active_=false`, `paused_=false` 로 전체 리셋하고 `pending_path_` 까지 비운다. PAUSE는 `active_` 를 유지하고 인덱스를 보존한다는 점이 다르다. 콜백들은 `active_` (STOP)와 `paused_` (PAUSE)를 각각 먼저 검사해 늦게 도착한 결과를 올바르게 무시/보류한다.

8. 향후 과제 / 주의사항

- **상태 피드백 역방향 채널:** 현재 Nav2 → command_server 로의 실시간 진행률/도착 상태 환류는 mission_control_node 기준 로깅 위주다. 태블릿 표시용 mission_status 동기화는 command_server 자체 상태에 의존한다.
 - **PAUSE 중 cmd_vel:** goal cancel 방식이므로 Nav2가 즉시 정지 명령을 내보내지만, 컨트롤러 감속 프로파일에 따라 완전 정지까지 짧은 지연이 있을 수 있다.
 - **테스트 입력 경로:** `/mission_input (WaypointList)` 직접 입력 경로는 디버그용이며 PAUSE/RESUME 흐름과 무관하다.
-

본 문서의 다이어그램 소스: `combat_nav2/docs/*.mmd` · 구현:

`combat_robot_nav2/src/mission_control_node.cpp` · 프로토콜:

`robot_server/include/command_server_protocol.hpp`